
VeriPy: Verified Python with LLM-Written Specifications and Proofs

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We present VeriPy, a verification toolchain built for coding agents. The agent
2 writes ordinary Python with `#@` specification comments that CPython ignores, and
3 the toolchain generates verifiable code from it, today in Dafny, with a Lean 4
4 backend planned behind the same thin intermediate representation. The same specs
5 also compile to runtime contracts hunted by a symbolic executor, so one surface
6 earns concrete counterexamples before any proof exists and machine-checked
7 proofs after. Proof material lives in an additions-only sidecar behind a whitelist
8 validator, so an agent can help prove but never assume, and verification failures
9 arrive as a versioned machine-readable taxonomy. On VeriPy-Bench, a curated
10 14-task corpus with a deterministic 63-mutant panel, Claude tiers spanning two
11 model generations place at 11–94 % on proof repair and at 63–77 % against a 78 %
12 curated ceiling on spec strength, and the two exams disagree about which tier is
13 strongest. Opus 5 posts the best repair column measured (17/18, including the task
14 no earlier tier restored) yet writes weaker specs than Sonnet 4.6. A raw-Dafny
15 baseline arm is designed to measure what the scaffolding contributes over handing
16 the model the prover’s input directly. Scoring these exams honestly was the hard
17 part. Mutation-scored specifications are established, but the refuter changes what
18 the score means. Adjudicating the same panel by witness-carrying execution and
19 by the prover, we find strictly one-directional disagreement and show that a bare
20 prover “Killed” verdict conflates proof-apparatus breakage and safety failures with
21 genuine spec refutation. From eleven silent scoring defects, including one that
22 flooded a vacuous spec at 38 % rather than 0 % and an answer-key retrieval that
23 recurred across vendors, we distill three rules, led by publishing a trivial-spec floor
24 beside every reported rate.

25 1 Introduction

26 Agents are fluent in Python and not in proof languages, and VeriPy takes the position that they should
27 not have to be. The agent writes ordinary Python with `#@` specification comments, and the toolchain
28 owns the translation to a proof language, generating verifiable Dafny today with a Lean 4 backend
29 planned behind the same thin intermediate representation. Everything downstream is built for a model
30 in the loop. Feedback is machine-readable rather than prose, proof edits are confined to a sidecar the
31 toolchain can police, and the evaluation is designed to survive the model gaming it.

32 We make three contributions.

- 33 • **The system** (Section 2). VeriPy verifies a fragment of typed Python with specs as `#@`
34 comment contracts, compiled both to runtime contracts and to Dafny. Proof material is
35 confined to an additions-only sidecar validated by a whitelist (ghost declarations with bodies

```

#@ verified
#@ ensures result == (exists n in range(len(operations) + 1) :: sum(operations[:n]) < 0)
def below_zero(operations: list[int]) -> bool:
    balance = 0
    for i in range(len(operations)):
        #@ invariant balance == sum(operations[:i])
        #@ invariant forall n in range(i + 1) :: sum(operations[:n]) >= 0
        #@ proof SliceSnoc(operations, i)
        balance += operations[i]
        if balance < 0:
            return True
    return False

// task.proofs.dfy (the additions-only sidecar)
lemma SliceSnoc(s: seq<int>, i: int)
    requires 0 <= i < |s|
    ensures PySlice(s, 0, i + 1) == PySlice(s, 0, i) + [s[i]]
{
}

```

Figure 1: VeriPy-Bench task `below_zero` (HumanEval/3). The postcondition quantifies over prefix sums. Z3 does not connect the preamble’s clamped slice semantics to the one-element extension unaided, so the `@ proof` clause invokes `SliceSnoc` from the sidecar. The lemma body is empty because Dafny proves the statement once it is stated. The whitelist validator accepts the sidecar because it contains only a bodied ghost declaration.

only), so agent edits can help prove but never assume. Failures are a versioned machine-readable taxonomy. The fragment is a generation target rather than a filter on found code. By an optimistic upper-bound survey, 65 % of HumanEval and 67 % of MBPP canonical solutions fall inside it, against 3–23 % of functions in nine typed OSS libraries.

- **VeriPy-Bench and its exams** (Section 3). A curated 14-task corpus, verified to full assurance height, drives two exams. Proof repair restores a deleted, provably load-bearing lemma pack. Spec writing annotates spec-stripped code, scored on the same 63-mutant panel as the curated reference. Six Claude tiers across two model generations are placed, with intervals at trial and task level. Repair columns span 11 % to Opus 5’s 94 % (17/18, including 3/3 on the task no earlier tier restored), while on spec strength five tiers place at 63–77 % against a 78 % curated ceiling and the generations invert, with Opus 5 at 66 % under Sonnet 4.6’s 75 %. A raw-Dafny baseline arm hands the model the prover’s input directly, with no whitelist and raw verifier errors, to isolate what the scaffolding contributes (Table 4).
- **Honest scoring** (Section 4). Mutation-scoring specs is established for contracts under KeY [1] and Dafny [2], but the refuter changes what the score means. Adjudicating our panel by a witnessed refuter and by the prover, agreement is perfect on the cleanly refutable class (49/49), disagreement is strictly one-directional (12 prover-only kills, 0 witnessed-only), and mechanical classification shows none of the 12 is a pure spec refutation. We distill three rules from eleven measured scoring defects, ten from our own instrument’s construction, the newest a cross-vendor answer-key retrieval in which Grok 4.6 behind the Cursor agent CLI defeated its read-only mode, its sandbox flag, and a wildcard tool-deny list.

2 VeriPy

Design positions. VeriPy is a shallow embedding. The prover never hosts a deep model of Python. A typed fragment is lowered directly to the target language under a semantics-pinning preamble, and the lowering is validated against CPython (below) rather than trusted. Four positions follow.

- **Single source of truth.** The annotated Python file is the only human-maintained artifact. Specs live in the source as comments, the generated Dafny model is regenerated on every run and never hand-edited, and the one durable proof artifact is the additions-only sidecar. There is no second codebase to keep in sync.
- **The source is the deployed artifact.** Translation-based verification leaves the verified model in the prover’s language, where an application cannot import it. Here the verified function is the ordinary Python source itself, imported directly with no adapter layer, and the

69 annotations cost nothing at runtime because CPython ignores comments (runtime contract
70 checking is an opt-in mode).

- 71 • **The ecosystem survives, and adoption is brownfield.** Annotated modules remain ordinary
72 Python, so test runners, type checkers, linters, and virtual environments work unchanged,
73 with no parallel prover build infrastructure entering the project. Verification is opted into per
74 function, so critical invariants earn proofs while surrounding glue code stays unannotated.
- 75 • **The backend is a target, not an identity.** Python semantics are pinned once in the lowering
76 catalog rather than re-derived per prover. The encoder sits behind a thin backend-neutral
77 intermediate representation. Dafny is the shipped backend, a Lean 4 emitter is planned over
78 the same representation, and identical tasks would then carry a second prove and fidelity
79 column.

80 **Spec surface.** Specifications are comments beginning `#@`, so annotated modules remain ordinary
81 Python that CPython runs unchanged (Figure 1). The clause grammar covers an opt-in `verified`
82 marker, `requires`, `ensures`, `loop invariant` and `decreases`, and a ghost proof clause that
83 invokes a named sidecar lemma at a program point. Spec expressions are Python expressions
84 extended with `==>`, `<==>`, and bounded `forall/exists` over finite domains, plus `result` and
85 `old()` in postconditions, a deliberate choice that keeps every spec executable. A `requires` or
86 `ensures` clause therefore has two compiled meanings, a runtime check and a Dafny obligation, while
87 `invariant`, `decreases`, and `proof` feed only the prover.

88 **Pipeline.** A front end enforces fragment conformance (an AST conformance pass plus strict static
89 typing, backed by an encoder that hard-fails on any construct without a lowering rule), and the
90 encoder emits Dafny against a versioned preamble that pins Python semantics the SMT solver would
91 otherwise get wrong, such as floor division, the floor-style `%` (Dafny’s is Euclidean), clamped slices,
92 and truthiness. The pipeline separates concerns into four gate stages, namely conformance, types,
93 encoding, and proof. The encoder is the largest trusted component, so it is validated rather than
94 argued about. A translation-validation harness compiles the Dafny model back to executable form
95 and differentially tests it against CPython on property-based inputs. The runtime rail compiles the
96 same specs to contracts hunted by a symbolic executor, so a spec earns concrete counterexamples
97 before any proof exists. That pre-proof regime is exactly where LLM-written specs live, and it is why
98 the scoring question of Section 4 matters.

99 **Agent interface.** Everything an agent touches is constrained and machine-readable. Proof material
100 lives in an additions-only sidecar file beside the source. A whitelist validator admits only bodied
101 ghost declarations and rejects axioms, bodiless lemmas, `assume`, attributes, and non-declaration
102 content, so a sidecar can help prove but cannot introduce assumptions. Verification outcomes are
103 structured payloads under a versioned failure taxonomy that names the failure kind (prover obligations
104 such as postcondition, invariant, bounds, and termination, sidecar resolution errors, and front-end
105 conformance rejections) with source positions mapped back to Python lines. The embedding API
106 never prints, never exits, and returns expected failures as values, properties pinned by tests. The
107 repair loop feeds the taxonomy back to the engine, which may edit only the sidecar.

108 **The fragment is a generation target.** An AST-level survey with optimistic method matching (an
109 upper bound) accepts 3–23 % of functions across nine typed OSS libraries, but 107 of 164 HumanEval
110 (65.2 %) and 650 of 974 MBPP (66.7 %) canonical solutions. Library plumbing was never the v1
111 audience. The kind of function LLMs are asked to write, and that specs naturally describe, is
112 overwhelmingly fragment-shaped, and code outside the fragment is refused at conformance rather
113 than scored.

114 3 VeriPy-Bench and the exams

115 Each task climbs an assurance ladder made of parse and conformance, the runtime contract hunt,
116 deterministic mutant-panel scoring [8], encoding, proof, and fidelity checks. The rungs catch different
117 defect classes (Table 2). The fidelity rung’s unique-catch cell is empty on this corpus, admitted rather
118 than filled. The 14 tasks (Table 1) carry curated specifications and, where the proof needs lemmas,
119 curated sidecars, all verified to full ladder height. On the 63-mutant panel the curated specs refute

Table 1: Corpus provenance. Benchmarks translated from existing suites are standard practice (Verus-Bench translates Diffy, MBPP, and CloverBench [16]). The graded artifacts (specs, sidecars, panels, whitelist) are project-original. HumanEval is MIT-licensed, and per-task license metadata rides in the artifact.

Source	Tasks	Sidecar-bearing	Notes
HumanEval (ported)	11	5	implementations maximally memorized
Project-original	3	1	incl. one MBPP-style task
Total	14	6	63-mutant deterministic panel

Table 2: The ladder’s rungs and the measured catch unique to each on this corpus.

Rung	Measured catch the other rungs missed
gate (parse, conformance)	2 of 42 weakest-tier spec-writing trials named a symbol outside the spec grammar (<code>pow</code>) and were rejected before scoring, counted 0/ N
hunt	every clean kill carries a concrete counterexample input, which prover kills of the same mutants never do (Section 4)
panel	the vacuous spec clears every other rung at full height and kills 0/63, and only this rung scores it
encode	2 of 63 panel mutants leave the encodable fragment, so the prover cannot judge them at all
prove	12 spec-writing trials matched or exceeded curated kill rates yet failed to prove, strength the panel certifies and provability it cannot
fidelity	(none measured yet)

120 49 (78 %), a figure two independent code paths (the ladder’s own panel stage and the exam scorer)
 121 compute separately and agree on. The remainder is *not* spec weakness. Thirteen mutants crash
 122 concrete execution before any contract can be witnessed and one diverges, and under refutation-only
 123 crediting (Section 4.2) those never count. The per-task refutation spread (0–100 %) is driven entirely
 124 by crash share, with zero clean survivors. The instrument separates a floor (vacuous specs score 0 %)
 125 from a ceiling (curated specs score 78 %), and engine-written specs populate the range between.

126 **Proof-repair exam.** Delete the sidecar. The engine must restore verification within four iterations
 127 of structured feedback. Admission requires the pack to be *provably load-bearing*, meaning deleting it
 128 breaks the proof. This is the same criterion AutoVerus applied in excluding the 26.9 % of DafnyBench
 129 tasks needing no proof annotations [16]. Eight of our fourteen tasks prove from invariants alone
 130 and are excluded, and six qualify. The screen itself must be able to fail. An early version asserted
 131 breakage the encoder already guaranteed (deleted lemmas still referenced), so it could not fail for
 132 any task. It now strips the proof clauses too and requires a genuine prover failure, with an explicit
 133 inconclusive verdict.

134 **Spec-writing exam.** Strip every specification. The engine annotates bare code with no verification
 135 feedback (grammar-rejected proposals are retried, at most three per trial), under a freeze validator
 136 enforcing AST-equality of the comment-stripped source. AutoVerus reports disciplining models into
 137 not modifying the implementation as an open prompting problem [16], and we mechanize the control.
 138 The result is scored on the *same* panel as the curated specs. Anti-gaming is built in, since a vacuous
 139 spec reaches full ladder height and kills 0/63.

140 **Raw-Dafny baseline.** To isolate what the scaffolding contributes, a baseline arm hands the model
 141 exactly what the prover sees, the encoder’s Dafny module with implementation and specifications
 142 frozen by construction, and asks for the proof file free-form. No whitelist, no taxonomy, and feedback
 143 is raw Dafny output. The same verifier judges both sides, which is what makes the comparison
 144 honest (a baseline that hand-translates the Python could verify a mistranslation). Accepted baseline
 145 answers are audited for assumption smuggling (assume, axiom attributes, bodiless lemmas), and
 146 audited-unsound successes are reported separately rather than credited.

147 **Protocol.** Engines run headless with all tools denied, in an empty working directory, under a wall
 148 clock, with pinned task ordering, bounded prompts, and the probe discipline of Section 4.2. This

Table 3: Proof repair over 6 tasks $\times$ 3 trials per arm. The full arm allows up to 4 iterations with structured verifier feedback, and the one-shot arm a single attempt. Brackets give 95 % Wilson intervals. The task-level column uses the clustering-honest unit (majority of 3 trials restored, $n=6$). The first four tiers carry per-row resolved model ids in their ledgers (claude-haiku-4-5-20251001, claude-sonnet-4-6, claude-opus-4-8[1m], claude-fable-5), and the Claude 5 tiers are pinned by invocation (-model recorded verbatim per call). All columns ran at a 600 s engine wall except Sonnet 5, whose 600 s attempt lost 4 of 36 cells to the wall (unmeasured, not failed) and whose column was therefore re-measured in full at a disclosed 1800 s wall. One Sonnet 5 cell died to a transient CLI failure and is counted in the denominator (10/17 measured, 59 %). Loop deltas are descriptive, and no delta interval excludes zero.

Engine	Full loop	One-shot	Loop Δ	Task-level
Haiku 4.5	2/18 = 11% [3,33]	2/18 = 11% [3,33]	+0	1/6 [3,56]
Sonnet 4.6	4/18 = 22% [9,45]	4/18 = 22% [9,45]	+0	1/6 [3,56]
Opus 4.8	9/18 = 50% [29,71]	6/18 = 33% [16,56]	+3	3/6 [19,81]
Fable 5	14/18 = 78% [55,91]	12/18 = 67% [44,84]	+2	5/6 [44,97]
Sonnet 5	10/18 = 56% [34,75]	5/18 = 28% [12,51]	+5	4/6 [30,90]
Opus 5	17/18 = 94% [74,99]	12/18 = 67% [44,84]	+5	6/6 [61,100]
Grok 4.6 (out-of-family)	15/18 = 83% [61,94]	15/18 = 83% [61,94]	+0	6/6 [61,100]

Table 4: Baseline decomposition on proof repair (headline engines). The raw-Dafny arm removes the whitelist and the taxonomy and feeds raw Dafny output. Cells marked (*running*) are being measured under the pinned protocol and freeze before submission. Unsound baseline “successes” (assumption smuggling) are reported in the audit row, never credited.

	Opus 4.8	Fable 5
Raw Dafny, one-shot	(<i>running</i>)	(<i>running</i>)
Raw Dafny, loop (raw errors)	(<i>running</i>)	(<i>running</i>)
VeriPy one-shot	6/18 = 33% [16,56]	12/18 = 67% [44,84]
VeriPy full loop	9/18 = 50% [29,71]	14/18 = 78% [55,91]
Audited-unsound raw successes	(<i>running</i>)	(<i>running</i>)

149 is *retrieval*-hardening, not contamination-hardening. With tools enabled, an agent once located
150 the curated sidecar, which is public, and returned it verbatim, a retrieval result wearing a proof-
151 completion costume. The same class recurred, across vendors, against a stack of nominal tool denials
152 (Section 4.2). Memorization of the *implementations* is assumed and stated. Five of six repair-roster
153 functions are HumanEval ports, yet scores span 11–94 % on identical tasks, and the roster’s hardest
154 task (rolling_max, HumanEval/9) is itself maximally memorized yet resisted every tier of the
155 original grid (0/6 against Fable 5) before falling to Opus 5’s loop. Pack overlap is quantified rather
156 than assumed away. Two pilot runs independently derived 8- and 5-lemma gcd packs against the
157 curated 8-lemma pack, sharing only the entry-point lemma the frozen proof clauses name (earlier-
158 instrument runs, reported as narrative). Recall of implementations demonstrably does not purchase
159 proof success. Unlike the never-public Verus proofs behind AutoVerus’s leakage argument [16], our
160 curated sidecars *are* public. We disclose that and deny retrieval mechanically instead. AutoVerus’s
161 authors-familiar-with-the-tasks disclosure is the kin of our curated-authorship disclosure (Section 5).

162 **Repair results.** Table 3 places the tiers across the range. The 4-iteration budget bought Haiku 4.5
163 and Sonnet 4.6 nothing (+0 each), Opus 4.8 +3 of 18, Fable 5 +2, and Opus 5 +5, consistent
164 with feedback helping only models strong enough to act on it, stated descriptively (no delta interval
165 excludes zero). Where the loop pays, the mechanism is visible in Fable 5’s two failing tasks.
166 below_zero fails at *postconditions*, well-formed packs the prover can criticize, so feedback is
167 informative and the loop converts 0/3 one-shot into 2/3, recovering at iterations 3–4. rolling_max
168 dies *before* the prover for Fable 5 (conformance and resolution errors, nothing semantic to iterate on)
169 and went 0/6 across its arms. Feedback pays only where failures reach the prover, and Opus 5 is the
170 demonstration in the other direction. Its packs reach the prover, so the loop converts rolling_max
171 from 0/3 one-shot to 3/3 and below_zero from 0/3 to 2/3, the largest loop delta measured. Trials are
172 $k=3$ because a single run is not a rate. Between two earlier single runs of this roster, one task flipped

Table 5: Spec writing over all 14 tasks with $k=3$ trials, scored on the same 63-mutant panel as the curated reference (189 mutant-trials, with Opus 4.8 not run on this exam). Kills are refutation-only. The floor row is a single deterministic run (63 mutants), and engine and curated rows aggregate 3 trials. Task-level means use equal task weights and a t -interval with $n=14$. The 75/77/78 top cluster is statistically indistinguishable, placed rather than ranked.

Spec author	Killed	Rate	Validity	Task-level mean [95% CI]
floor (vacuous)	0/63	0%	14/14	0%
Haiku 4.5	119/189	63%	40/42	63% [39, 86]
Opus 5	124/189	66%	39/42	63% [42, 84]
Sonnet 5	133/189	70%	40/42	69% [47, 91]
Sonnet 4.6	141/189	75%	42/42	71% [50, 93]
Fable 5	145/189	77%	42/42	72% [51, 93]
curated reference	147/189	78%	14/14	73% [52, 95]
Grok 4.6 (out-of-family)	139/189	74%	42/42	71% [50, 92]

from restored-in-two-iterations to an attempt the prover times out on, in a direction no configuration change explains, so run-to-run variance is on the order of the effects measured. At task level the intervals are wide by construction and only the extremes separate, and we state this rather than pooling it away.

Spec-writing results. Table 5 shows the instrument separates the floor from everything and, weakly and at trial level only, the weakest tier from the top cluster. Within-cluster ranking is not licensed and not claimed. Dynamic range is real at task grain. Intermediate rates exist (Haiku 4.5 gcd 42 %), and modp separates weak from strong (Haiku 4.5 0 %, Sonnet 4.6 62 %, Fable 5 88 %, matching curated at 88 %). The strongest engine is near-deterministic, with identical per-task kill counts across all three trials on 13 of 14 tasks. The strength $\neq$ provability divergence is measured, not asserted. Across $n=12$ trials, engine-written specs matched or exceeded curated kill rates at *lower* ladder height. These are strong specs that do not prove, precisely the regime where witnessed scoring is the only scoring available. The Claude 5 mid-tiers sharpen the point by inverting across the exams. Opus 5 posts the strongest repair column measured (17/18) yet places at 66 % on spec strength, under Sonnet 4.6’s 75 %, and Sonnet 5 places at 70 %. Both sit below the 75–78 top cluster at trial level and overlap it at task level, so this is placement rather than ranking, but the direction is the instrument’s thesis in miniature. Within the Sonnet pair the pattern repeats. Sonnet 5 repairs at 56 % over Sonnet 4.6’s 22 % yet writes specs at 70 % under its 75 %. Proof strength and spec strength move independently, here across model generations.

Guard telemetry. In the original four-tier grid the sidecar whitelist rejected 6 of 236 proposals, all six from Fable 5 (five non-declaration, one forbidden token), and the rejected-gcd trial still restored after rejection feedback. The Claude 5 columns drew 19 rejections across 198 proposals (14 from Sonnet 5, 5 from Opus 5), so rejection pressure rises with model generation. The guard is load-bearing, not decorative. All columns ran on a single workstation, and engine API cost was between \$3 and \$33 per column. Full telemetry and token/cost accounting ride in the artifact.

4 Scoring spec strength honestly

4.1 Two refuters, one panel

Prover-adjudicated mutation scoring of contracts is prior art [1, 2]. What is not established is what each available refuter actually measures. Our leading observation is structural. **Prover-judged scoring requires a baseline that verifies**, because mutation is detected as “the proof stopped going through,” so the unmutated program must prove (all fourteen of ours do). LLM-written specifications in the pre-proof regime cannot supply that precondition. **Witnessed scoring does not need it**, because a kill is a concrete counterexample against the mutated program’s contracts, whether or not anything proves. Beneath the structural point, the two verdicts mean different things. A witnessed kill carries an input. It is sound but incomplete, since it under-credits strength where execution crashes first. A

Table 6: Dual adjudication of the 63-mutant panel, paired per mutant. The witnessed refuter is symbolic execution over compiled contracts with a counterexample required. The prover refuter is Dafny on the mutated program with curated specs and sidecar. Two out-of-fragment mutants the prover could not judge are excluded (61 shown).

	Prover kill	No prover kill
Witnessed kill	49	0
No witnessed kill	12	0
Prover-only kills, by obligation kind		<i>n</i>
proof-apparatus (hint asserts, fn. preconditions)		7
safety VC (bounds)		1
termination		1
multi-obligation (postcondition + others)		3

208 prover kill carries no witness, and proof brittleness (the mutant broke the lemma structure, not the
 209 spec) inflates it. We measured the difference by adjudicating the identical panel both ways (Table 6).
 210 Three findings, in order of importance.

- 211 1. **Perfect agreement on the cleanly refutable class.** All 49 witnessed kills are also prover
 212 kills, and no mutant survives both refuters. The scored panel has no clean survivors and no
 213 spec blind spots (equivalent mutants are adjudicated out of the denominator before scoring,
 214 following IronSpec’s relevance filtering [3]). Two further mutants fell outside the encodable
 215 fragment and the prover could not judge them at all.
- 216 2. **Disagreement is strictly one-directional and concentrated where concrete execution**
 217 **fails first.** We predicted witnessed kills the prover would miss. That cell is *empty*. The
 218 12 prover-only kills sit exactly on the crash/timeout class, which is close to a corollary of
 219 refutation-only crediting, and we say so. The empirical content is the empty cell and the
 220 kind decomposition.
- 221 3. **A bare pass/fail prover verdict conflates kill classes.** We reclassified the 12 prover-only
 222 kills *mechanically*. Apply the mutant, run the standard pipeline, and take the per-failure
 223 obligation kinds from the toolchain’s versioned failure taxonomy. A mutant with kinds in
 224 more than one category counts once, as multi-obligation. The result is 7 proof-apparatus,¹
 225 1 safety-VC, 1 termination, and 3 multi-obligation, and **none is a pure specification**
 226 **refutation.** Our own earlier hand-count had labeled four of them “genuine spec refutations”.
 227 The mechanical method dissolved that cell, an instance of precisely the conflation this
 228 section measures. Binary “Killed” verdicts (the scoring in [2]) cannot see this structure. The
 229 witnessed refuter separates crash from refutation natively.

230 Stated fairly, the refuters are complementary, not ranked. The prover reached verdicts concrete
 231 execution could not (crash-first paths, symbolic timeout), but its unique reach here is almost entirely
 232 apparatus- and safety-shaped.

233 4.2 The honesty protocol

234 Protocol first, confession second. Each rule is a failure class, the invariant that excludes it, and the
 235 probe that detects its violation.

236 **Rule 1 (Floor).** Vacuous specs pass every stage (vacuity has a long ancestry [9, 10]), and a crash
 237 credited as a kill is the implicit-oracle class in disguise [7]. *Invariant.* Publish the trivial-spec floor
 238 beside every rate. *Probe.* The floor arm itself, which must score 0.

239 **Rule 2 (Denominators).** If a failing answer leaves the denominator, failing becomes profitable. A
 240 spec refuted at the runtime hunt once contributed 0/0, vanishing from its own aggregate. *Invariant.*
 241 Subject-independent denominators. Every attempted task is scored over the *curated* panel, so a failed

¹One of the seven surfaced as an unmapped kind. Dafny’s “function precondition could not be proved” phrasing fell through the taxonomy and was resolved from the attached raw message. The gap is itself an instrument finding, fixed in a taxonomy-versioned update.

Table 7: Eleven silent scoring defects, by the direction the score lied and the rule whose probe class caught it. Each survived until probed, and none announced itself.

Defect instance	Direction	Rule
Interpreter crashes credited as kills, flooring a vacuous spec at 38 %	too good	1
Divergence credited as a kill (same hole, timeout bucket)	too good	1
Runtime-refuted spec vanished from its own denominator (0/0), so failing beat a weak spec	too good	2
Working-directory race let concurrent verifications certify each other’s code	both	3
Equivalent-mutant exclusions silently missed the panel after a coordinate change	too bad	3
Prompt renderer scored every spec-writing cell as engine error	too bad	3
Answer-key retrieval, where a tool-bearing agent returned the curated sidecar verbatim	too good	3
Resume logic hid failed cells from the aggregate	too good	3
Matrix status scoped to stale ledger rows, not the current roster	too bad	3
Report printed a rounded median (4.5 rendered as 4), not the computed one	too bad	3
Cross-vendor false zero, an extraction bug masking answer-key retrieval (below)	both	3

242 answer is $0/N$, never excluded. *Probe*. An intentionally failing arm must lower, never raise, the
243 aggregate.

244 **Rule 3 (Probes).** A harness bug is indistinguishable from a hard benchmark. *Invariant*. Run false-
245 perfect *and* false-zero probes before trusting any score. *Probe*. A known-good arm must score high,
246 and a known-broken one must score 0 for the right reason.

247 The rules are backed by eleven defect instances measured on this instrument (Table 7), each caught by
248 the probe class its row names, and every one produced a plausible number until it was probed. This is
249 not a private bug diary. The one published instrument whose scoring we could inspect, MutDafny [2],
250 reports a binary “Killed” with no why-classification, the exact conflation the previous subsection
251 measures. The conflation class caught us as well, when our own hand-classification of prover-only
252 kills briefly manufactured a “genuine spec-refutation” cell that the mechanical method dissolved.

253 **The eleventh instance, told in full.** It shows these are field-wide failure classes, not artifacts of
254 one harness. The out-of-family engine column, Grok 4.6 behind the Cursor agent CLI (resolved id
255 cursor-grok-4.6-high-fast), scored 0/36 on proof repair, with all 126 attempts dead before the
256 prover at the encode stage. Interrogating the zero (Rule 3) found two colliding causes. First, the CLI
257 interleaves tool-narration prose with its answer, and our reply extraction, pinned to Claude’s fencing,
258 passed the prose straight into the pipeline (“top-level ‘I’ll’ is not a ghost declaration”). Second,
259 beneath it, the disqualifier. The CLI’s machine-wide code-search tool escaped its read-only ask mode,
260 its sandbox flag, and every tool-deny variant we tried, including the wildcard. The raw transcript
261 narrates the escape (“Grep can see the files even when Read cannot”) and then returns the curated
262 gcd sidecar verbatim. The extraction bug masked the retrieval bug. Fix extraction first and the
263 column ships retrieval-inflated. Ship it as measured and it is a false zero. Neither is a measurement.
264 The holding control is an OS-level read-deny jail around the agent process, probe-verified in both
265 directions, with the agent remaining functional and its own search returning denied on the curated
266 trees. Three lessons generalize. Answer-key retrieval is not one vendor’s behavior, “read-only agent
267 mode” is not retrieval-safe, and tool-name deny lists leak.

268 **The remediated column, and why it matters.** Under the jail, with extraction fixed for both the
269 sidecar and the annotated-source answer shapes, the same engine produced a valid out-of-family
270 column (Tables 3 and 5). Grok 4.6 restored 15 of 18 repair trials in each arm (83 %) and reached
271 the same 74–77 % spec cluster as the strongest in-family tiers, with all 97 calls carrying consistent
272 engine-claimed provenance and zero whitelist rejections. The pre-flight gate re-verified retrieval-
273 blindness before every run. Two facts make this the answer to the natural “a Claude-built instrument
274 measuring Claude” objection. The out-of-family engine places inside the same range as the family,
275 not off it, so the instrument is not calibrated to one vendor. And it restored *every* repair task by

majority, including `rolling_max`, the prefix-maximum monotonicity proof no Claude tier below Opus 5 could reliably construct (Table 5 0/6). We confirmed the `rolling_max` pack was constructed, not retrieved. It is not byte-identical to the curated sidecar (different checksum, shorter proof bodies for the same source-mandated lemma names) and it verifies independently. Roster difficulty therefore reflects the proof, not the prover’s family.

5 Limitations

VeriPy verifies a fragment, and several planned components (boundary guard generation among them) are design, not shipped code. The claims here are about the built pipeline the exams exercise. The corpus is small, at fourteen tasks with four thin panels. $k=3$ is small and task-clustered, so trial-level intervals are anti-conservative. We report task-level numbers beside them, and the 11→94 gradient is instrument validation, never an engine ranking (adjacent intervals overlap at both levels). The in-family engines are Claude capability tiers, run at default reasoning effort (recorded as a condition), with resolved model identities pinned per row for the original grid and by recorded invocation for the Claude 5 columns. The curated references were authored with same-family LLM assistance under human direction and adversarial review, so distance-to-curated is a within-family measure and roster feasibility was implicitly defined by what that family can prove. One out-of-family column (Grok 4.6) tests that concern directly and places inside the family’s range while restoring every repair task, so the roster is not calibrated to one vendor. One out-of-family engine is not a cross-vendor study, and its provenance is engine-claimed rather than per-call verified (the CLI does not report a served model), a disclosed weakness relative to the Claude columns. The measurements themselves are author-blind (fixed mechanical panel, prover adjudication). Memorization of implementations is assumed, not ruled out (Section 3 controls). Saturation risk is owned. Opus 5 reaches 17/18 on repair, so this roster’s repair tier is close to solved, but the corpus is an instrument, not a challenge. An all-hard roster would have no resolution across today’s range and could not distinguish “cannot” from “harness broken” (Rule 3). Spec strength still shows range (63–77 against the 78 ceiling, with no engine matching the curated panel), and the growth path is a two-tier roster, this resolution tier plus a frontier tier (prefix-aggregate lemmas, dynamic programming pending encoder support, nonlinear arithmetic, packs requiring new helper functions), plus the planned Lean 4 backend’s second prove and fidelity column over identical tasks. The fragment ceiling is a routing decision, not a coverage claim. Programs outside the encodable fragment are refused at conformance rather than scored. The venue is non-archival, and ledgers and pinned invocations ship in the artifact.

6 Related work

Verified Python. Nagini [11] verifies statically typed Python by translation to Viper, and its strict-typing front-end discipline is precedent for our type gate. Strata-Python [13], the Python front end of AWS’s Strata verification platform, translates Python through the Laurel intermediate verification language to SMT backends, and VeriPy’s thin backend-neutral IR keeps such a second emitter possible. VeriPy differs from both in compiling one comment-contract surface to runtime contracts and Dafny alike, in the additions-only whitelist-validated proof sidecar, and in treating the agent interface and its scoring as first-class. The runtime rail builds on CrossHair [12], whose symbolic execution over contracts is our witnessed refuter.

Mutation-scored specifications. Knüppel et al. [1] score JML contract completeness by mutation under KeY. MutDafny [2] does the same against the Dafny verifier (40 operators, 118,458 mutants, 82 % average score) with a binary Killed verdict, excluding Invalid and TimedOut mutants from the score as we do. IronSpec [3] mutates the *specification* and pioneers automated mutation-relevance filtering, precedent for our adjudication. Groce et al. [4] use mutants to probe what verification establishes, and mCoq [5] brings mutation to Coq proofs. The oracle-quality line (Jahangirova et al. [6]) is the test-oracle-shaped ancestor, and Barr et al. [7] name the implicit-oracle (crash-crediting) class our Rule 1 instance exhibits.

LLM-written specifications. The nl2postcond line [14] scores generated postconditions by bug-catching power, proof-free. FMCAD’24 [15] applies symbolic-test metrics to generated specs for verification-aware languages. Our addition is the per-mutant *pairing* of witnessed and prover verdicts on one surface, and the measured strength≠provability divergence.

Proof synthesis and benchmarks. AutoVerus [16] generates Verus proofs (137/150 on Verus-Bench, itself translated from Diffy/MBPP/CloverBench) and grounds three of our protocol positions, namely benchmark translation as standard practice, the never-public-proofs leakage argument, and the no-proof-needed exclusion our load-bearing screen mechanizes. DafnyBench [17] evaluates proof synthesis at scale with whole-error-message feedback. Clover [18] checks consistency among code, specs, and docs, and dafny-annotator [19] learns to insert Dafny annotations.

References

- [1] A. Knüppel, L. Schaer, I. Schaefer. How much specification is enough? Mutation analysis for software contracts. *FormaliSE@ICSE*, 2021.
- [2] I. Amaral, A. Mendes, J. Campos. MutDafny: A mutation-based approach to assess Dafny specifications. *ICSE*, 2026. arXiv:2511.15403.
- [3] E. Goldweber, W. Yu, S. A. Vakil-Ghahani, M. Kapritsos. IronSpec: Increasing the reliability of formal specifications. *OSDI*, 2024.
- [4] A. Groce, I. Ahmed, C. Jensen, P. E. McKenney, J. Holmes. How verified (or tested) is my code? Falsification-driven verification and testing. *Autom. Softw. Eng.* 25(4), 2018.
- [5] A. Celik, K. Palmkog, M. Parovic, E. J. Gallego Arias, M. Gligoric. Mutation analysis for Coq. *ASE*, 2019.
- [6] G. Jahangirova, D. Clark, M. Harman, P. Tonella. Test oracle assessment and improvement. *ISSTA*, 2016.
- [7] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo. The oracle problem in software testing: a survey. *IEEE TSE* 41(5), 2015.
- [8] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, M. Harman. Mutation testing advances: an analysis and survey. *Advances in Computers* 112, 2019.
- [9] I. Beer, S. Ben-David, C. Eisner, Y. Rodeh. Efficient detection of vacuity in temporal model checking. *FMSD* 18(2), 2001.
- [10] O. Kupferman, M. Vardi. Vacuity detection in temporal model checking. *STTT* 4(2), 2003.
- [11] M. Eilers, P. Müller. Nagini: A static verifier for Python. *CAV*, 2018.
- [12] P. Schanely. CrossHair: An analysis tool for Python that blurs the line between testing and type systems. Software, <https://github.com/pschanely/CrossHair>, accessed August 2026.
- [13] Strata-Python: Python language support for the Strata verification platform. Software, <https://github.com/strata-org/Strata-Python>, accessed August 2026.
- [14] M. Endres, S. Fakhoury, S. Chakraborty, S. K. Lahiri. Can large language models transform natural language intent into formal method postconditions? *PACMSE/FSE*, 2024.
- [15] S. K. Lahiri. Evaluating LLM-driven user-intent formalization for verification-aware languages. *FMCAD*, 2024. arXiv:2406.09757.
- [16] C. Yang, X. Li, M. R. H. Misu, J. Yao, W. Cui, Y. Gong, C. Hawblitzel, S. K. Lahiri, J. R. Lorch, Shuai Lu, F. Yang, Z. Zhou, Shan Lu. AutoVerus: Automated proof generation for Rust code. *PACMPL* 9 (*OOPSLA2*), 2025. arXiv:2409.13082.
- [17] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, M. Tegmark. DafnyBench: A benchmark for formal software verification. arXiv:2406.08467, 2024.
- [18] C. Sun, Y. Sheng, O. Padon, C. W. Barrett. Clover: Closed-loop verifiable code generation. *SAIV*, 2024. arXiv:2310.17807.
- [19] G. Poesia, C. Loughridge, N. Amin. dafny-annotator: AI-assisted verification of Dafny programs. arXiv:2411.15143, 2024.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and the bulleted contributions in Section 1 state exactly the three delivered contributions (the VeriPy system of Section 2, VeriPy-Bench with its two exams and the raw-Dafny baseline arm of Section 3, and the honest-scoring analysis of Section 4), with the scope caveats (fourteen tasks, single vendor family for the headline tiers, instrument validation rather than engine ranking) carried in Section 5. Numbers still being measured (the Sonnet 5 and Opus 5 columns of Table 3 and the raw-Dafny arm of Table 4) are marked (*running*) rather than claimed as results, and they freeze before submission.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: A dedicated Limitations section (Section 5) covers corpus size, thin panels, task-clustered $k=3$ trials whose trial-level intervals are anti-conservative (task-level numbers reported beside them), single-vendor-family engines, curated-reference authorship, the memorization assumption, and saturation of the top tier.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper contains no theorems or formal theoretical results. All claims are empirical measurements on a fixed deterministic mutant panel.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The mutant panel is deterministic, the exam, baseline, and adjudication protocol (headless engines, tool denial, empty working directory, bounded prompts, iteration budgets, $k=3$ trials, mechanical kind-classification) is specified in Section 3–Section 4, and the artifact carries append-only ledgers and pinned invocations for every reported cell. Two tables carry cells still being measured under the identical pinned protocol (the Sonnet 5 and Opus 5 columns of Table 3 and the raw-Dafny arm of Table 4), marked (*running*) in the paper. Those cells freeze before submission, and their ledgers and pinned invocations join the artifact at that point.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.

- (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An artifact accompanies the submission with the toolchain, corpus, mutant panels, exam harness, append-only ledgers, and the pinned commands that regenerate each table (Section 3, Section 5). Engine columns depend on commercial model APIs, so the ledgers include the recorded transcripts, allowing verification of every reported cell without re-querying the engines.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: No training is performed. Section 3 specifies exam construction, admission screens, iteration budgets, freeze validation, trial counts, and the retrieval-hardening protocol, and Section 5 records that engines ran at default reasoning effort. Resolved per-row model identifiers and full run configuration are pinned in the artifact ledgers.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.

- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Every reported rate carries a 95% interval, with Wilson intervals at trial level and clustering-honest task-level units beside them (majority-of-3 Wilson at $n=6$, equal-weight task-mean t -intervals at $n=14$), and with the anti-conservativeness of trial-level intervals under task clustering stated explicitly (Tables 3 and 5, Section 5). No ranking claim is made where intervals overlap. The Sonnet 5 and Opus 5 columns announced in Table 3 and the cells marked (*running*) in Table 4 carry no rates yet, and both receive the identical interval treatment when they freeze before submission.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Compute is modest. All verifier and symbolic-execution runs execute on a single workstation, and the engine columns are API calls costing between \$3 and \$33 per column. Per-call token and cost accounting rides in the artifact (Section 3). No GPU training or cluster compute is involved.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research involves no human subjects, no personal or crawled data, and no dual-use release. The venue's double-blind submission mechanics are followed (anonymous author block, third-person self-reference), and the work conforms to the Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This is foundational measurement methodology for scoring formal specifications. Honest scoring of verification instruments is the (positive) point, and we see no direct path to negative applications, so the paper does not include a separate societal-impact section.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No models are trained or released and no scraped data is distributed. The released artifact is a small curated benchmark and scoring harness for program verification, which poses no misuse risk requiring safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Original creators are credited and the ported HumanEval tasks are identified in Table 1, whose caption states the HumanEval license (MIT) and that per-task license metadata rides in the artifact. The underlying tools and prior benchmarks are cited.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The project-original assets (curated specifications and sidecars, the deterministic mutant panels, the whitelist validator, and the exam harness) are described in Section 2–Section 3 and ship in the artifact together with append-only ledgers, pinned invocations, and per-cell documentation.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no research with human subjects. All exam subjects are LLM engines.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects were involved, so IRB or equivalent review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are the objects of study (the engine tiers that take the exams of Section 3), and their vendor family, default reasoning-effort setting, and role in authoring the curated references (same-family LLM assistance under human direction and adversarial review) are all disclosed in Section 5.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.